CHRISCO Youth Aflame
CHRISCO DIGITAL ACADEMY | CHRISCO FELLOWSHIP OF CHURCHES

SWE & LLM Mastery Course Handbook

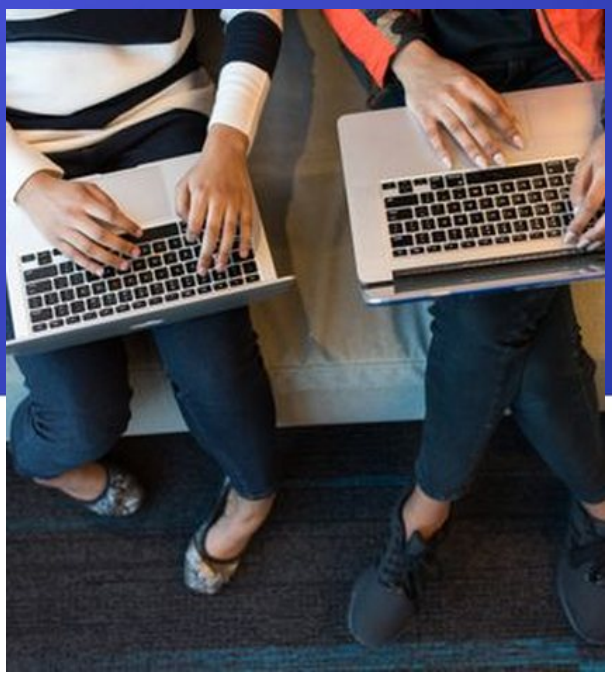
"Build real, AI-powered software with modern engineering."



Software Engineering & Applied Systems Track | 10 Weeks | Intermediate | 28-Page Illustrated Edition

Prepared by CHRISCO Youth Aflame | chriscoyouthaflame2025@gmail.com

Course Foreword



We are living through a rare shift: large language models (LLMs) have made it possible to build genuinely intelligent software faster than ever. Combine solid software engineering with the ability to harness LLMs, and you become the kind of builder companies are desperate to hire, or the founder who ships products alone.

This book assumes you can already code a little (or are learning). It focuses on how to think like an engineer and how to build with AI, not on one specific language. The principles here will outlast any single tool or model.

THE PRACTITIONER PROMISE

You will learn software engineering foundations, how large language models work, prompt and API skills, and how to design, build, test and deploy an AI-powered application of your own.

The goal is a working AI-powered app you build yourself. Nothing proves skill, to employers, clients, or yourself, like a real thing that works. Build as you read.

Published by CHRISCO Digital Academy under CHRISCO Youth Aflame (*CHRISCO Fellowship of Churches*), this 28-page course handbook explains every module clearly with visual diagrams, real East African case studies, and step-by-step labs. Reach our mentorship desk anytime at chriscoyouthaflame2025@gmail.com.



Key Learning Outcomes

01

Software engineering basics: Engineering = reliable, maintainable, problem-solving code.

02

Prompt engineering: Prompts are part of your app's code.

03

AI-powered app development: Use the LLM as one component of solid engineering.

04

Real-world projects: Ship one complete, live AI-powered app.

Across 10 Weeks of structured practice, each of the 8 modules is given a dedicated two-page spread: first unpacking the concept visually, then walking through a real field scenario and hands-on lab.

By Module 08, you will have built and verified a complete production software feature ready to serve employers, clients, ministries, and communities with excellence.

Table of Contents

Course Foreword & Key Outcomes 02

01 Foundations & Core Mindset (Modules 1 to 2) 05

02 Core Frameworks & Strategy (Modules 3 to 4) 10

03 Execution & Applied Mastery (Modules 5 to 6) 15

04 Scaling & Capstone Project (Modules 7 to 8) 20

05 Complete Syllabus & Fellowship Reflections 25

06 90-Day Action Plan & Stewardship 27



01
Foundations & Core Mindset

Module 01: Software engineering basics | Module 02: Understanding LLMs

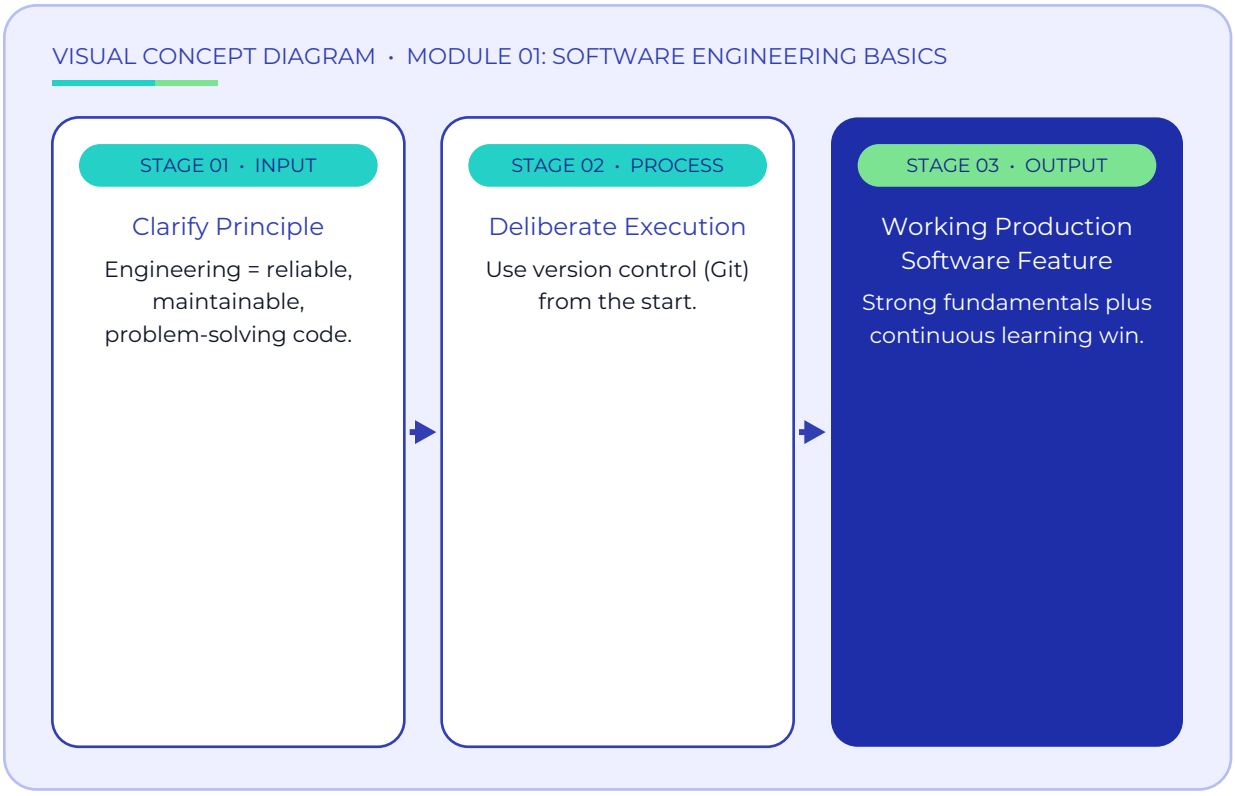
MODULE 01 · PART 1

1.1 Software engineering basics

Software engineering is more than writing code, it's building software that is reliable, maintainable and solves real problems. That means breaking problems into clear parts, writing clean and understandable code, using version control (like Git) to track changes, and thinking about how pieces fit together. These habits separate a hobbyist from a professional engineer.

Key foundations include understanding data structures and logic, reading and reusing others' code, debugging systematically, and collaborating using tools like Git and GitHub. You don't need to know everything, you need strong fundamentals and the ability to learn continuously, because technology changes fast. Good engineering thinking is the durable skill; specific tools come and go.

Why This Principle Matters in Depth: Many learners read about *software engineering basics* at a surface level, yet struggle when building a real production software feature. Notice the underlying logic: when you anchor your work around the rule that engineering = reliable, maintainable, problem-solving code., you remove guesswork and build repeatable confidence. Take time to study the visual concept diagram below before moving to the worked case study on the next page.





MODULE 01 | WORKED CASE STUDY

Real-World Scenario:
Two coders build the same feature. One writes messy code that only they understand; the other writes clean, organised code with clear names and version control. Six months later, only the second can easily fix and extend it, that's engineering, not just coding.

How to Apply This Lesson Step by Step

Look closely at why the example above succeeded: instead of waiting for perfect conditions, the practitioner focused on one concrete deliverable and tested it in the real world. Use the guided assignment below to build the exact same muscle in your own SWE & LLM Mastery workbook.

HANDS-ON LAB ASSIGNMENT · MODULE 01

Your Practical Task: Set up Git and a GitHub account. Put a small piece of code (even a simple script) into a repository, making at least two commits.

Self-Check Before Moving On: Write down what worked, what felt unclear, and one specific improvement you will make on your second repetition. Competence comes from completed reps, never passive reading.

Module 01 Key Takeaways

01

Engineering = reliable, maintainable, problem-solving code.

02

Use version control (Git) from the start.

03

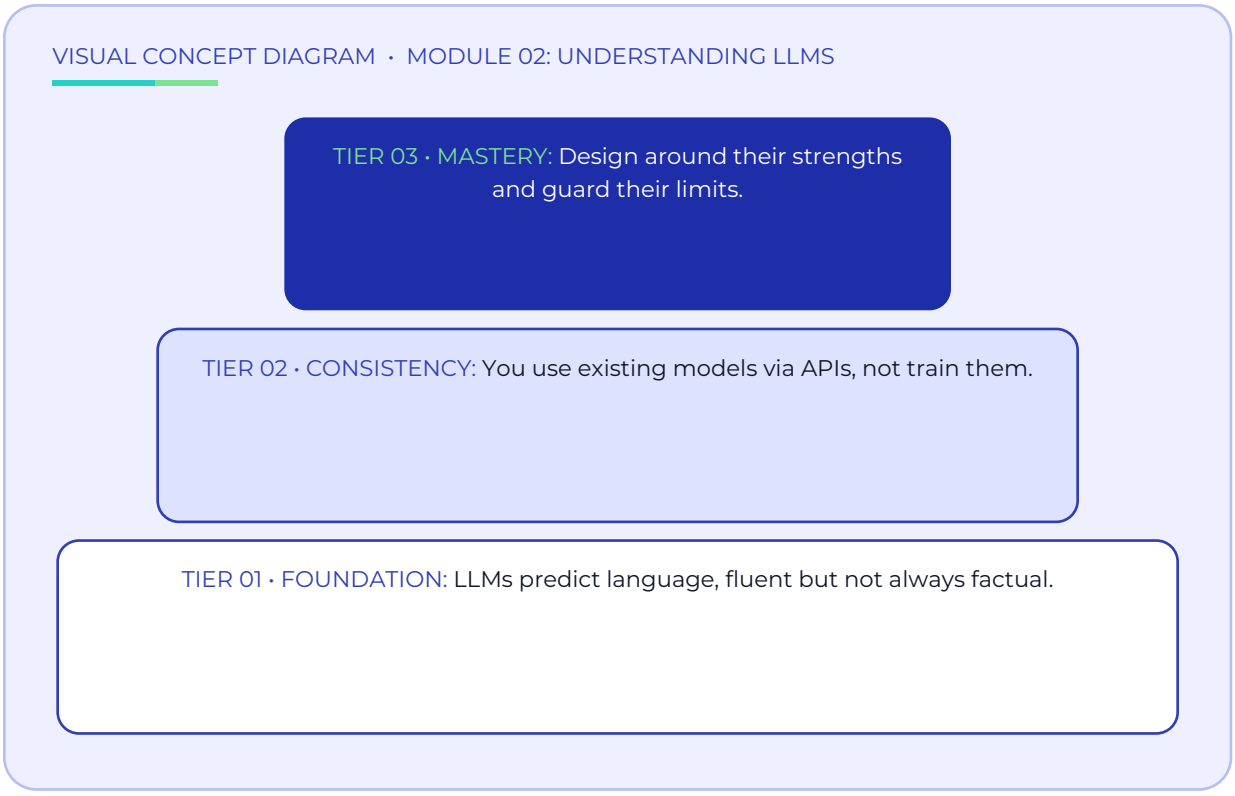
Strong fundamentals plus continuous learning win.

1.2 Understanding LLMs

Large language models are AI systems trained on vast amounts of text to predict and generate human-like language. They can write, summarise, translate, answer questions, and even help write code. Crucially, understand what they are: powerful pattern-predictors, not databases of facts, which is why they can be fluent yet occasionally confidently wrong ('hallucinate').

As a builder, you don't need to train these models yourself, you use existing ones (like those from OpenAI, Anthropic, Google and open-source options) via APIs. Knowing their strengths (language tasks, reasoning help, code assistance) and limits (accuracy, up-to-date facts, cost per use) lets you design smart applications that use LLMs where they shine and guard against where they fail.

Why This Principle Matters in Depth: Many learners read about *understanding llms* at a surface level, yet struggle when building a real production software feature. Notice the underlying logic: when you anchor your work around the rule that llms predict language, fluent but not always factual., you remove guesswork and build repeatable confidence. Take time to study the visual concept diagram below before moving to the worked case study on the next page.



CHRISCO Youth Aflame | chriscoyouthaflame2025@gmail.com

MODULE 02 | FIELD BRIEF

Focus Skill:
Understanding LLMs

Core Rule:
LLMs predict language, fluent but not always factual.

Target Output:
Working production software feature

MODULE 02 | WORKED CASE STUDY

Real-World Scenario:
A developer building a customer-support tool used an LLM to draft helpful replies but added a step to pull real, verified answers from the company's own documents, combining the model's fluency with reliable facts to avoid confident wrong answers.

How to Apply This Lesson Step by Step

Look closely at why the example above succeeded: instead of waiting for perfect conditions, the practitioner focused on one concrete deliverable and tested it in the real world. Use the guided assignment below to build the exact same muscle in your own SWE & LLM Mastery workbook.

HANDS-ON LAB ASSIGNMENT · MODULE 02

Your Practical Task: Use an LLM (via a chat interface) for three different tasks: writing, summarising, and helping with code. Note where it excelled and where it erred.

Self-Check Before Moving On: Write down what worked, what felt unclear, and one specific improvement you will make on your second repetition. Competence comes from completed reps, never passive reading.

Module 02 Key Takeaways

01

LLMs predict language, fluent but not always factual.

02

You use existing models via APIs, not train them.

03

Design around their strengths and guard their limits.



02
Core Frameworks & Strategy

Module 03: Prompt engineering | Module 04: Building with APIs

MODULE 03 · PART 1

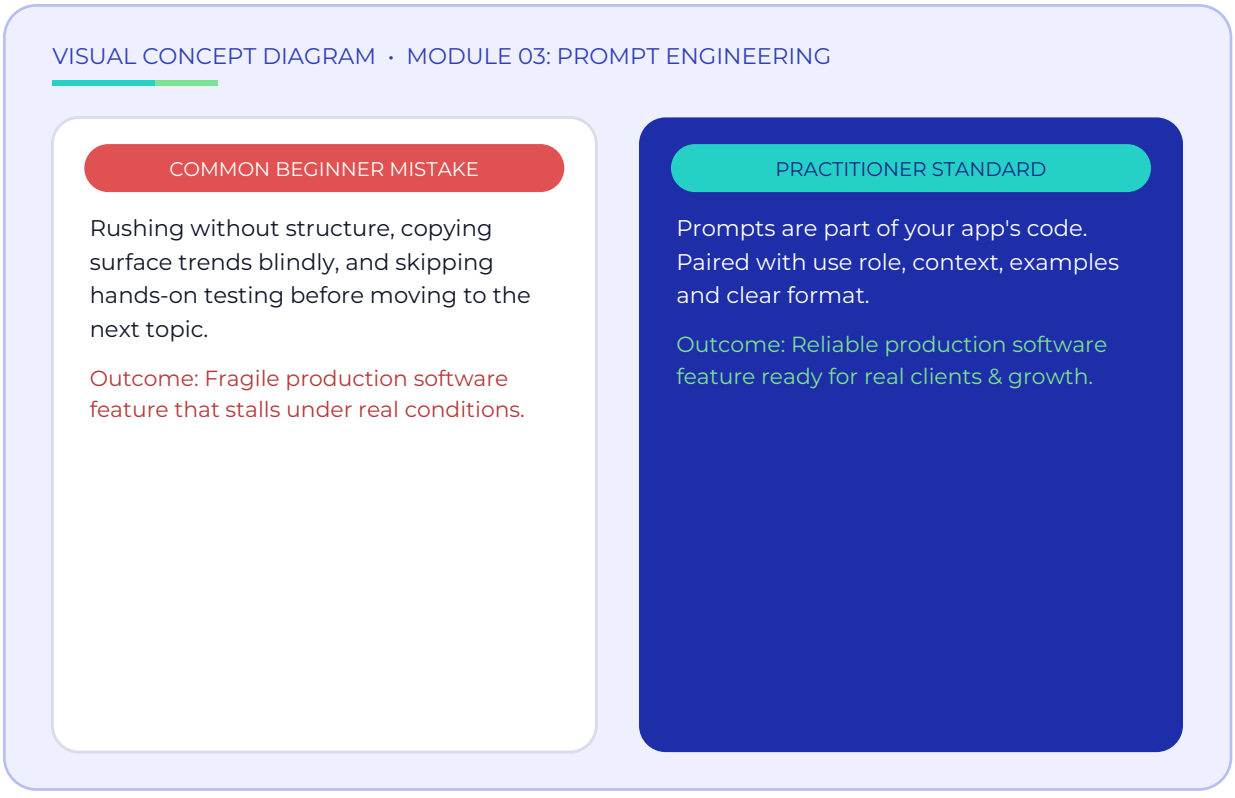
2.1 Prompt engineering

Prompt engineering is the skill of instructing an LLM to get reliable, high-quality output, essential when building AI apps. A strong prompt provides clear context, a specific task, the desired format, and often examples. In applications, prompts are part of your code: well-crafted, tested prompts are the difference between an AI feature that delights users and one that frustrates them.

Techniques include giving the model a role, breaking complex tasks into steps, providing examples of the desired output (few-shot prompting), and instructing it to reason step by step for harder problems. You also learn to handle and validate the model's responses in code. Treat prompts as you would any code: iterate, test with many inputs, and refine until they're robust.

Why This Principle Matters in Depth: Many learners read about *prompt engineering* at a surface level, yet struggle when building a real production software feature. Notice the underlying logic: when you anchor your work around the rule that prompts are part of your app's code., you remove guesswork and build repeatable confidence. Take time to study the visual concept diagram below before moving to the worked case study on the next page.

VISUAL CONCEPT DIAGRAM · MODULE 03: PROMPT ENGINEERING



MODULE 03 | FIELD BRIEF

Focus Skill:
Prompt engineering

Core Rule:
Prompts are part of your app's code.

Target Output:
Working production software feature

MODULE 03 | WORKED CASE STUDY

Real-World Scenario:
In an app that categorises customer messages, the developer's first prompt gave inconsistent results. By adding a clear role, three examples, and a strict output format, the categorisation became reliable enough to ship, the prompt itself was the fix.

How to Apply This Lesson Step by Step

Look closely at why the example above succeeded: instead of waiting for perfect conditions, the practitioner focused on one concrete deliverable and tested it in the real world. Use the guided assignment below to build the exact same muscle in your own SWE & LLM Mastery workbook.

HANDS-ON LAB ASSIGNMENT · MODULE 03

Your Practical Task: Write a prompt that reliably makes an LLM output a specific format (e.g. always return JSON with certain fields). Test it on several different inputs.

Self-Check Before Moving On: Write down what worked, what felt unclear, and one specific improvement you will make on your second repetition. Competence comes from completed reps, never passive reading.

Module 03 Key Takeaways

01

Prompts are part of your app's code.

02

Use role, context, examples and clear format.

03

Iterate and test prompts against many inputs.

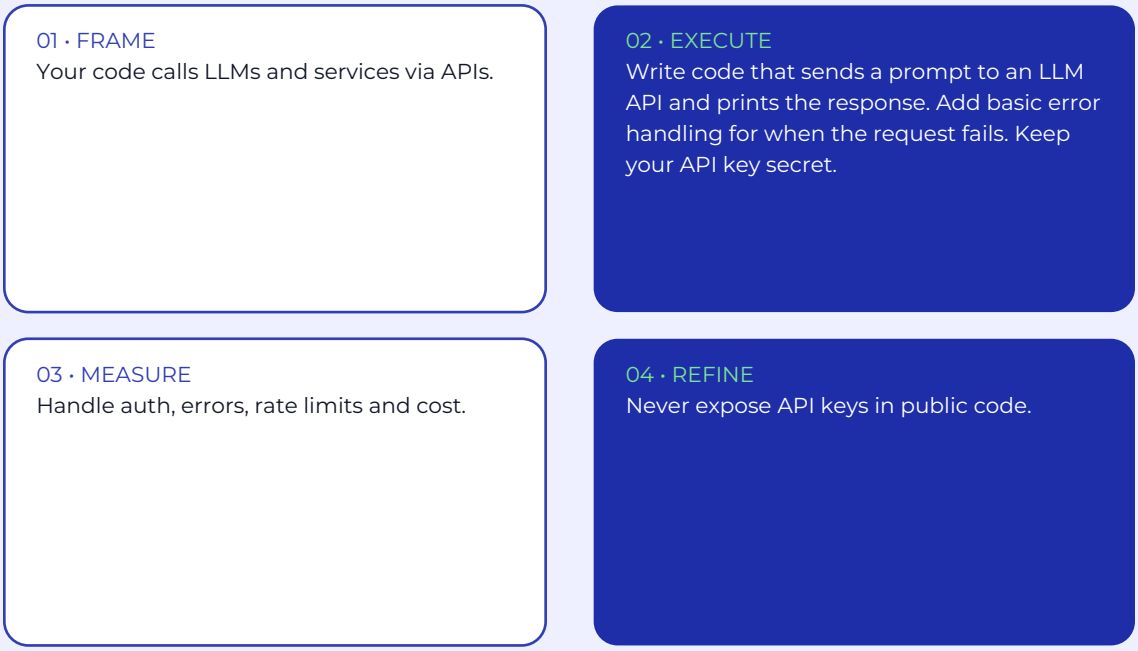
2.2 Building with APIs

APIs let your software talk to LLMs and other services programmatically. Instead of a person typing into a chat, your code sends a request (with your prompt and settings) to the model's API and receives a response to use in your app. Learning to authenticate, send requests, handle responses, and manage errors and rate limits is the core skill for building AI-powered software.

You'll also work with other APIs, databases, payment, external data, since real apps combine many services. Key professional concerns include keeping API keys secret (never in public code), handling failures gracefully, and being mindful of cost (LLM APIs charge per use). Mastering API integration is what lets you assemble powerful applications from existing building blocks.

Why This Principle Matters in Depth: Many learners read about *building with apis* at a surface level, yet struggle when building a real production software feature. Notice the underlying logic: when you anchor your work around the rule that your code calls llms and services via apis., you remove guesswork and build repeatable confidence. Take time to study the visual concept diagram below before moving to the worked case study on the next page.

VISUAL CONCEPT DIAGRAM · MODULE 04: BUILDING WITH APIS



MODULE 04 | FIELD BRIEF

Focus Skill:
Building with APIs

Core Rule:
Your code calls LLMs and services via APIs.

Target Output:
Working production software feature

MODULE 04 | WORKED CASE STUDY

Real-World Scenario:
A developer built a study-helper app where the user's question is sent via API to an LLM, and the answer is displayed in the app. The whole 'intelligence' came from an API call, he focused on the experience around it, not building a model.

How to Apply This Lesson Step by Step

Look closely at why the example above succeeded: instead of waiting for perfect conditions, the practitioner focused on one concrete deliverable and tested it in the real world. Use the guided assignment below to build the exact same muscle in your own SWE & LLM Mastery workbook.

HANDS-ON LAB ASSIGNMENT · MODULE 04

Your Practical Task: Write code that sends a prompt to an LLM API and prints the response. Add basic error handling for when the request fails. Keep your API key secret.

Self-Check Before Moving On: Write down what worked, what felt unclear, and one specific improvement you will make on your second repetition. Competence comes from completed reps, never passive reading.

Module 04 Key Takeaways

01

Your code calls LLMs and services via APIs.

02

Handle auth, errors, rate limits and cost.

03

Never expose API keys in public code.



03
Execution & Applied Mastery

Module 05: AI-powered app development | Module 06: Testing and deployment

MODULE 05 · PART 1

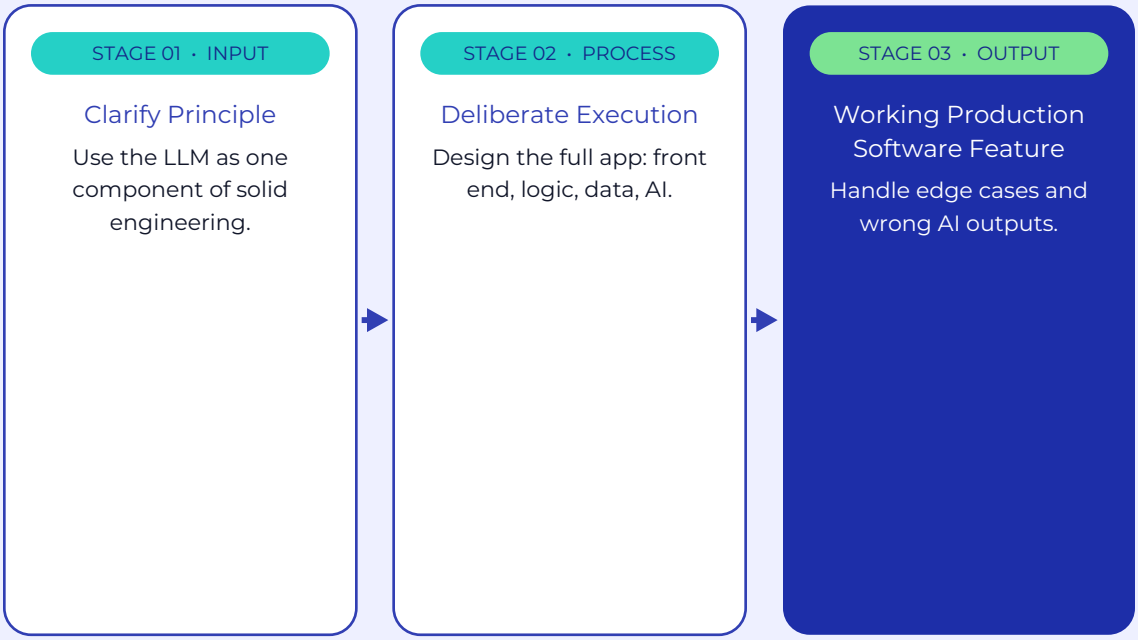
3.1 AI-powered app development

Now you combine engineering and LLMs into a real application. This means designing the app: what problem it solves, the user experience, the flow of data, and where the LLM fits. Good AI apps use the model as one component within solid engineering, with a clear interface, reliable data handling, and thoughtful prompts, rather than just a thin wrapper around a chatbot.

Think about architecture: a front end users interact with, back-end logic, data storage, and the LLM integration, all working together. Consider the user's journey and edge cases (what if the AI returns something wrong or nothing?). The best AI products feel seamless because good engineering surrounds and supports the AI, handling the messy reality the model alone can't.

Why This Principle Matters in Depth: Many learners read about *ai-powered app development* at a surface level, yet struggle when building a real production software feature. Notice the underlying logic: when you anchor your work around the rule that use the llm as one component of solid engineering., you remove guesswork and build repeatable confidence. Take time to study the visual concept diagram below before moving to the worked case study on the next page.

VISUAL CONCEPT DIAGRAM · MODULE 05: AI-POWERED APP DEVELOPMENT



MODULE 05 | FIELD BRIEF

Focus Skill:
AI-powered app development

Core Rule:
Use the LLM as one component of solid engineering.

Target Output:
Working production software feature

MODULE 05 | WORKED CASE STUDY

Real-World Scenario:
A team built an app that turns a user's rough notes into organised meeting minutes: the front end captures notes, the back end sends a carefully-prompted request to an LLM, validates the result, and saves it. Solid engineering around the AI made it genuinely useful.

How to Apply This Lesson Step by Step

Look closely at why the example above succeeded: instead of waiting for perfect conditions, the practitioner focused on one concrete deliverable and tested it in the real world. Use the guided assignment below to build the exact same muscle in your own SWE & LLM Mastery workbook.

HANDS-ON LAB ASSIGNMENT · MODULE 05

Your Practical Task: Design (on paper) an AI-powered app: the problem it solves, the user flow, where the LLM fits, and how you'll handle wrong or empty AI responses.

Self-Check Before Moving On: Write down what worked, what felt unclear, and one specific improvement you will make on your second repetition. Competence comes from completed reps, never passive reading.

Module 05 Key Takeaways

01

Use the LLM as one component of solid engineering.

02

Design the full app: front end, logic, data, AI.

03

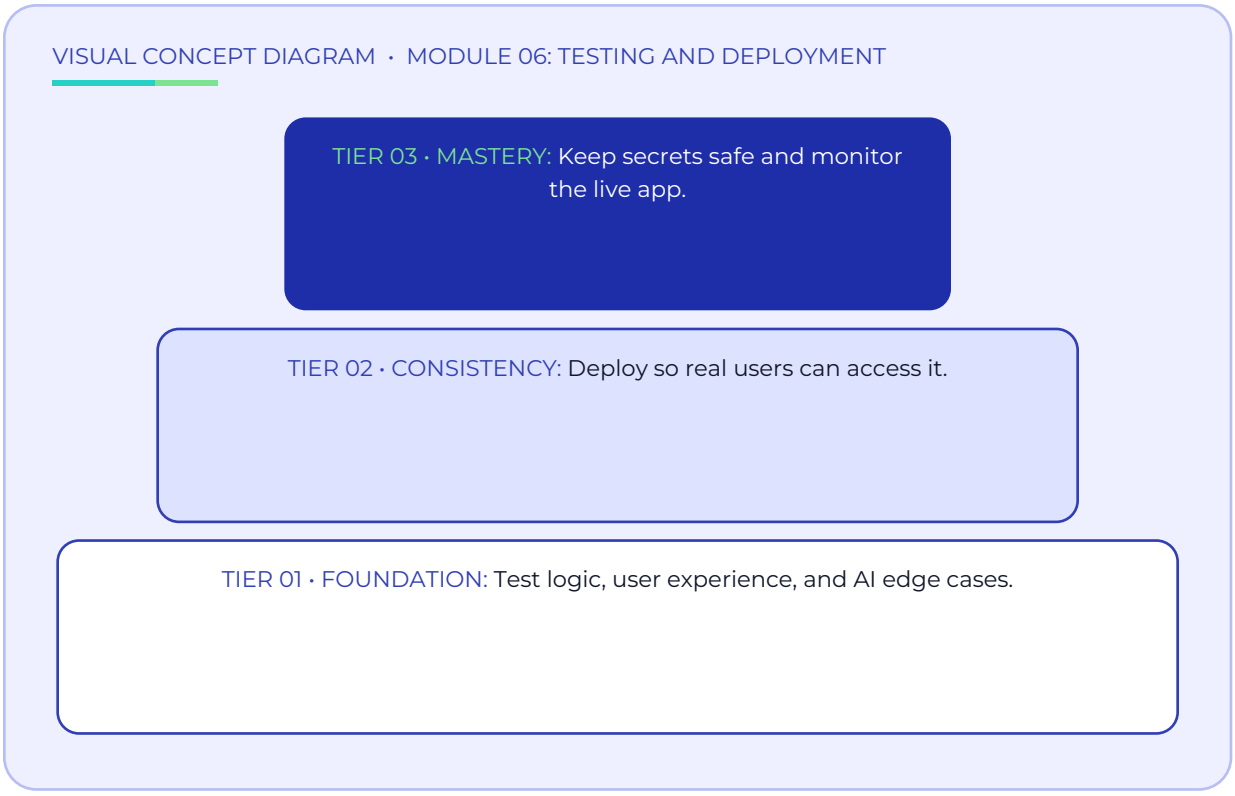
Handle edge cases and wrong AI outputs.

3.2 Testing and deployment

Software isn't done when it works on your machine, it must work reliably for users. Testing means checking that your app behaves correctly, including with unexpected inputs and when the AI misbehaves. Write tests for your logic, and manually test the whole user experience. With AI features, also test that your prompts and validation handle a wide range of inputs gracefully.

Deployment means putting your app where users can access it, on a hosting platform or cloud service. You'll learn to deploy your application, manage environment settings (like API keys securely), and monitor it once live. Shipping a real, deployed app, not just code on your laptop, is a major milestone and exactly what proves professional capability.

Why This Principle Matters in Depth: Many learners read about *testing and deployment* at a surface level, yet struggle when building a real production software feature. Notice the underlying logic: when you anchor your work around the rule that test logic, user experience, and ai edge cases., you remove guesswork and build repeatable confidence. Take time to study the visual concept diagram below before moving to the worked case study on the next page.



MODULE 06 | FIELD BRIEF

Focus Skill:
Testing and deployment

Core Rule:
Test logic, user experience, and AI edge cases.

Target Output:
Working production software feature

MODULE 06 | WORKED CASE STUDY

Real-World Scenario:
A developer's app worked perfectly in testing until a user entered something unexpected and the AI returned nonsense. Because he'd added validation and tested edge cases before deploying, the app handled it gracefully instead of crashing in front of real users.

How to Apply This Lesson Step by Step

Look closely at why the example above succeeded: instead of waiting for perfect conditions, the practitioner focused on one concrete deliverable and tested it in the real world. Use the guided assignment below to build the exact same muscle in your own SWE & LLM Mastery workbook.

HANDS-ON LAB ASSIGNMENT · MODULE 06

Your Practical Task: Test one part of your app with unusual and empty inputs and fix what breaks. Then deploy a simple app to a free hosting platform so it's live online.

Self-Check Before Moving On: Write down what worked, what felt unclear, and one specific improvement you will make on your second repetition. Competence comes from completed reps, never passive reading.

Module 06 Key Takeaways

01

Test logic, user experience, and AI edge cases.

02

Deploy so real users can access it.

03

Keep secrets safe and monitor the live app.



04
Scaling, Capstone & Excellence

Module 07: Real-world projects | Module 08: Build your AI app

MODULE 07 · PART 1

4.1 Real-world projects

Skill becomes credibility through real projects. Build things that solve genuine problems, for yourself, a community, or a small business, and put them on GitHub as your portfolio. Employers and clients care far more about what you've actually built than about certificates. A few solid, well-documented projects demonstrating engineering plus AI integration speak louder than any CV line.

Work on projects end-to-end: from idea and design through building, testing, deploying, and writing a clear README explaining what it does and how. Contributing to open-source or collaborating with others also builds real-world experience and network. Each project teaches new problems and grows your ability, and your body of work becomes your strongest career asset.

Why This Principle Matters in Depth: Many learners read about *real-world projects* at a surface level, yet struggle when building a real production software feature. Notice the underlying logic: when you anchor your work around the rule that real, deployed projects prove skill best., you remove guesswork and build repeatable confidence. Take time to study the visual concept diagram below before moving to the worked case study on the next page.

VISUAL CONCEPT DIAGRAM · MODULE 07: REAL-WORLD PROJECTS

COMMON BEGINNER MISTAKE

Rushing without structure, copying surface trends blindly, and skipping hands-on testing before moving to the next topic.

Outcome: Fragile production software feature that stalls under real conditions.

PRACTITIONER STANDARD

Real, deployed projects prove skill best. Paired with document projects clearly on github.

Outcome: Reliable production software feature ready for real clients & growth.

MODULE 07 | FIELD BRIEF

Focus Skill:
Real-world projects

Core Rule:
Real, deployed projects prove skill best.

Target Output:
Working production software feature

MODULE 07 | WORKED CASE STUDY

Real-World Scenario:
A self-taught developer landed a job not through a degree but through three deployed projects on GitHub, including an AI-powered tool, each with a clear README. The interviewer said the working, documented projects made the decision easy.

How to Apply This Lesson Step by Step

Look closely at why the example above succeeded: instead of waiting for perfect conditions, the practitioner focused on one concrete deliverable and tested it in the real world. Use the guided assignment below to build the exact same muscle in your own SWE & LLM Mastery workbook.

HANDS-ON LAB ASSIGNMENT · MODULE 07

Your Practical Task: Plan a portfolio project that solves a real problem using engineering + an LLM. Build it end-to-end and write a clear README for it on GitHub.

Self-Check Before Moving On: Write down what worked, what felt unclear, and one specific improvement you will make on your second repetition. Competence comes from completed reps, never passive reading.

Module 07 Key Takeaways

01

Real, deployed projects prove skill best.

02

Document projects clearly on GitHub.

03

Your body of work is your strongest asset.

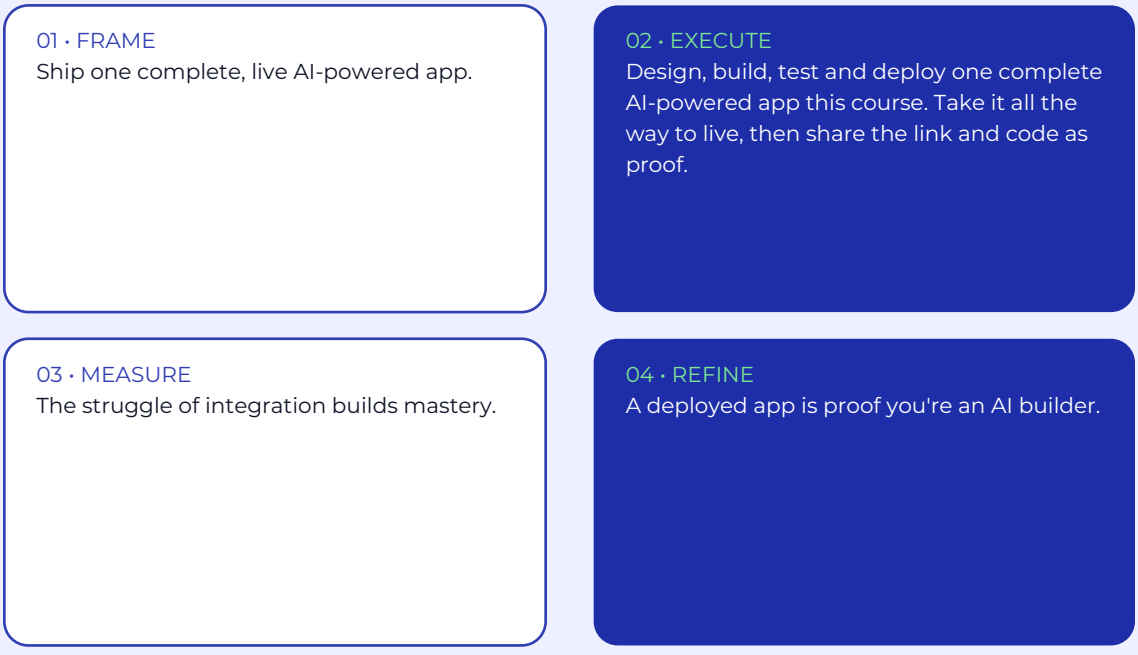
4.2 Build your AI app

Your capstone is a complete, working AI-powered application that you design, build, test and deploy yourself. Choose a real problem an LLM can genuinely help solve, keep the scope achievable, and take it all the way to a live, usable product. This ties together every skill in the book, engineering fundamentals, LLM understanding, prompting, APIs, app design, testing and deployment.

Expect challenges, integrating everything and handling real-world messiness is hard, and that struggle is exactly where mastery forms. Break it into steps, use documentation and AI assistants when stuck, and keep going until it runs for real users. When it's live, you'll have crossed from 'learning' to 'builder of AI software', with proof in hand and confidence earned.

Why This Principle Matters in Depth: Many learners read about *build your ai app* at a surface level, yet struggle when building a real production software feature. Notice the underlying logic: when you anchor your work around the rule that ship one complete, live ai-powered app., you remove guesswork and build repeatable confidence. Take time to study the visual concept diagram below before moving to the worked case study on the next page.

VISUAL CONCEPT DIAGRAM · MODULE 08: BUILD YOUR AI APP



MODULE 08 | FIELD BRIEF

Focus Skill:
Build your AI app

Core Rule:
Ship one complete, live AI-powered app.

Target Output:
Working production software feature

MODULE 08 | WORKED CASE STUDY

Real-World Scenario:
A learner built and deployed a small app that summarises long documents into key points using an LLM API, with a clean interface and error handling. It was modest but real, live, and useful, and it became the project that opened doors for her.

How to Apply This Lesson Step by Step

Look closely at why the example above succeeded: instead of waiting for perfect conditions, the practitioner focused on one concrete deliverable and tested it in the real world. Use the guided assignment below to build the exact same muscle in your own SWE & LLM Mastery workbook.

HANDS-ON LAB ASSIGNMENT · MODULE 08

Your Practical Task: Design, build, test and deploy one complete AI-powered app this course. Take it all the way to live, then share the link and code as proof.

Self-Check Before Moving On: Write down what worked, what felt unclear, and one specific improvement you will make on your second repetition. Competence comes from completed reps, never passive reading.

Module 08 Key Takeaways

01

Ship one complete, live AI-powered app.

02

The struggle of integration builds mastery.

03

A deployed app is proof you're an AI builder.

5.1 Complete 8-Module Practice & Competency Matrix

Review every module in SWE & LLM Mastery using the table below. Check off each module only after you have completed its hands-on assignment and verified your work against the core principle.

Module	Focus Topic	Core Principle	Workbook Check
Module 01	Software engineering basics	Engineering = reliable, maintainable, problem-solving code.	Completed [ ]
Module 02	Understanding LLMs	LLMs predict language, fluent but not always factual.	Completed [ ]
Module 03	Prompt engineering	Prompts are part of your app's code.	Completed [ ]
Module 04	Building with APIs	Your code calls LLMs and services via APIs.	Completed [ ]
Module 05	AI-powered app development	Use the LLM as one component of solid engineering.	Completed [ ]
Module 06	Testing and deployment	Test logic, user experience, and AI edge cases.	Completed [ ]
Module 07	Real-world projects	Real, deployed projects prove skill best.	Completed [ ]
Module 08	Build your AI app	Ship one complete, live AI-powered app.	Completed [ ]

CAPSTONE PORTFOLIO STANDARD

Before graduating from this handbook, combine all eight modules into one finished, shareable production software feature: *Design, build, test and deploy one complete AI-powered app this course. Take it all the way to live, then share the link and code as proof.* When ready for mentor feedback, email your work to chriscoyouthaflame2025@gmail.com.

5.2 Applied Fellowship Case Reflections

Real transformation happens when learners turn theory into consistent service. Reflect on these cohort case highlights from across East Africa as you prepare your own project:

"Two coders build the same feature. One writes messy code that only they understand; the other writes clean, organised code with clear names and version control. Six months later, only the second can easily fix and extend it, that's engineering, not just coding."

Cohort Case Reflection | Starting Strong in Module 01

STAGE 01

01

STAGE 02

02

"A developer built a study-helper app where the user's question is sent via API to an LLM, and the answer is displayed in the app. The whole 'intelligence' came from an API call, he focused on the experience around it, not building a model."

Mid-Course Breakthrough | Applying Strategy in Module 04

"A learner built and deployed a small app that summarises long documents into key points using an LLM API, with a clean interface and error handling. It was modest but real, live, and useful, and it became the project that opened doors for her."

Capstone Completion | Shipping Real Work in Module 08

STAGE 03

03

Stewardship Principle: "Whatever your hand finds to do, do it with excellence, integrity, and faithfulness so your skill becomes a blessing to many."

06 90-Day Action Roadmap

True mastery in SWE & LLM Mastery is measured by what you build, ship, and steward after finishing the lessons. Follow this structured 90-day implementation plan developed by CHRISCO Youth Aflame (under CHRISCO Fellowship of Churches) to turn study into fruitfulness.

6.1 Four-Stage Post-Course Milestones

01

Days 1 to 14: Foundation Sprint

Complete Modules 1 to 4 labs and document your baseline specimen in SWE & LLM Mastery.

02

Days 15 to 30: First Live Output

Complete Modules 5 to 8 and ship your first real capstone deliverable: Design, build, test and deploy one complete AI-powered app this course. Take it all the way to live, then share the link and code as proof.

03

Days 31 to 60: Feedback & Iteration

Share your work with a mentor, peer, or real client/community audience and refine weak points.

04

Days 61 to 90: Service & Multiplication

Use your skill to serve your church, business, or clients faithfully and mentor one younger learner.

Follow-Up Stage	CHRISCO Support Channel	Contact & Cadence
Workbook Review	CHRISCO Digital Academy Desk	chriscoyouthaflame2025@gmail.com
Capstone Submission	CHRISCO Youth Aflame Mentors	shambetz@gmail.com
Peer Accountability	CHRISCO Fellowship Study Hubs	Monthly Cohort Check-In

CHRISCO Youth Aflame
CHRISCO DIGITAL ACADEMY | CHRISCO FELLOWSHIP OF CHURCHES

Equipping Youth, Elevating Purpose

CHRISCO Youth Aflame
CHRISCO Digital Academy
Under CHRISCO FELLOWSHIP OF CHURCHES

Official Gmail Contacts
chriscoyouthaflame2025@gmail.com
shambetz@gmail.com

Official 28-Page Illustrated Course E-Book for SWE & LLM Mastery (Software Engineering & Applied Systems Track).
Prepared by CHRISCO Youth Aflame under CHRISCO FELLOWSHIP OF CHURCHES.
For course guidance or mentorship, email chriscoyouthaflame2025@gmail.com. CHRISCO Youth Aflame